Group 21 Progress Report:
My Group's Project Name

First Author, Second Author, Third Author
{parasutt,macid2,macid3}@mcmaster.ca

1 Introduction

Identifying which instrument produced a sound is harder than it seems. Instruments in the same family, such as the oboe and cor anglais or the violin and viola, have very similar timbres, and a single instrument can sound quite different depending on the pitch, dynamics, and articulation it is played with. Being able to identify an instrument from its sound alone would make recordings easier to search and catalogue, for example to find other recordings that feature the same instrument as a given composition.

In this project, we build a machine learning system that classifies orchestral instruments from short audio recordings. We frame this as a single-label, multi-class classification task over 19 brass, woodwind, and string instruments, using single-note clips from the Philharmonia Orchestra sound sample library (Philharmonia Orchestra, n.d.). Each clip is converted from a raw waveform into a time–frequency representation using the Fourier transform. From this, we either extract hand-crafted spectral features for classical models such as SVMs and random forests, or compute log-mel spectrograms as input to a small convolutional neural network.

The task has several challenges. Audio must first be processed with signal processing techniques before it can be used as model input. The model must learn timbre rather than shortcuts such as pitch or loudness, since some instruments (e.g., tuba, contrabassoon, and double bass) can be identified from pitch range alone. The dataset is also heavily imbalanced, with over 1,500 violin clips but fewer than 100 for some instruments. Finally, because the recordings are clean studio samples, a model trained on them may not generalize to real-world recordings.

2 Related Work

Instrument recognition is a well-studied problem in music information retrieval. Early work focused on isolated notes, as in our task, and relied on hand-crafted features. Eronen and Klapuri (2000) combined cepstral coefficients with temporal features describing a note's attack and decay, and Herrera-Boyer et al. (2003) surveyed the spectral and temporal features and classifiers (such as k-nearest neighbours, decision trees, and SVMs) used for classifying isolated instrument sounds. These works motivate our classical baseline of MFCC statistics, spectral descriptors, and attack time fed to an SVM or random forest.

More recent work has used larger datasets and deep learning. The NSynth dataset (Engel et al., 2017) contains over 300,000 four-second notes from roughly 1,000 instruments labelled by instrument family, although its main focus is audio synthesis rather than classification. The IRMAS dataset (Bosch et al., 2012) targets the harder problem of recognizing the predominant instrument in polyphonic music across 11 classes. On IRMAS, Han et al. (2017) showed that a convolutional neural network trained on mel spectrograms outperforms feature-based approaches, which motivates our log-mel spectrogram CNN. Finally, PANNs (Kong et al., 2020) are CNNs pretrained on large-scale general audio data that transfer well to other audio classification tasks, and could serve as a pretrained alternative if our small CNN is limited by the size of our dataset.

Our task is different from these, we are classifying orchestral instruments rather than their broad families, using clean single-note recordings. Making it more fine-grained than Nsynth's but simpler than polyphonic recognition. Because every instrument is recorded across many pitches and dynamics.

3 Dataset

We use the Philharmonia Orchestra sound sample library (Philharmonia Orchestra, n.d.), a freely available collection of recordings made by members of the Philharmonia Orchestra. The library is distributed as MP3 files grouped by instrument, and each file name records the instrument, pitch, duration, dynamic, and articulation of the clip (e.g., `flute_A4_025_forte_normal.mp3`). The instrument name serves as our class label, and the remaining fields let us analyse which factors affect our models, such as whether accuracy drops at extreme pitches or quiet dynamics, without any manual annotation.

We use 19 instruments from three families: four brass, eight woodwind, and seven string instruments. The strings include the plucked guitar, mandolin, and banjo as well as the bowed orchestral strings. Table 1 lists the number of clips and the pitch range of each instrument. While inspecting the data, we found that the cor anglais folder contains every recording twice, once under `cor-anglais` and once under its alternate name `english-horn`, and that the two copies are byte-identical. We remove these duplicates, since otherwise copies of the same recording could appear in both the training and test sets. This leaves 13,535 clips.

The recordings vary along several axes. Most clips are single notes with a duration of 0.25, 0.5, 1, or 1.5 seconds (11,485 clips), with fewer long and very long sustained notes (961 clips) and 1,089 short phrases. Dynamics range from pianissimo to fortissimo. About 84% of clips use the instrument's standard articulation, while the rest cover 49 other playing techniques such as pizzicato, staccato, trills, tremolo, and muted playing. Pitch ranges overlap heavily between instruments, but the contrabassoon and tuba reach down to A♯0 and the double bass to C1, below every other instrument in the dataset.

The class distribution is highly imbalanced. Violin has 1,502 clips, about twenty times as many as banjo (74), and the three plucked strings together make up less than 2% of the data. As a result, overall accuracy alone can be misleading, since a model can score well while performing poorly on the rarest classes.

Instrument	Clips	Pitch range
Brass		
French horn	652	A♯1–F5
Trombone	831	E2–F6
Trumpet	485	E2–E6
Tuba	972	A♯0–F4
Woodwind		
Bass clarinet	944	C2–C6
Bassoon	720	A♯1–G5
Clarinet	846	D3–C7
Contrabassoon	710	A♯0–D4
Cor anglais	691	E3–B5
Flute	878	C4–F7
Oboe	596	A♯3–A♯6
Saxophone	733	D3–G6
Strings		
Banjo	74	C3–E6
Cello	889	C2–C7
Double bass	852	C1–G4
Guitar	106	E2–E6
Mandolin	80	G3–A6
Viola	974	C3–F♯7
Violin	1,502	G3–E8
Total	**13,535**	

Table 1: Number of clips and pitch range for each instrument, after removing duplicate cor anglais recordings.

4 Features

Both of our models share the same preprocessing. Each MP3 file is decoded, converted to mono, and resampled to 16 kHz. We then trim leading and trailing silence and pad or crop every clip to a fixed length, which we will choose experimentally between 1 and 2 seconds. From this waveform, we compute a short-time Fourier transform, map its frequency bins onto the mel scale, which approximates how humans perceive pitch, and take the logarithm to compress the large range of energy between quiet and loud frequencies. The result is a log-mel spectrogram that shows how the energy at each frequency changes over the course of the note.

Classical baseline. For the SVM and random forest, we engineer a fixed-length feature vector for each clip. Since these models cannot handle a variable-length sequence of frames, we summarize each frame-level feature by its mean and standard deviation over time. The features, following Eronen and Klapuri (2000) and Herrera-Boyer et al. (2003), are:

- **MFCCs**, which describe the overall shape of the spectrum and are the main indicator of timbre;
- **spectral centroid and rolloff**, which measure

how bright a sound is;

- **spectral flatness and zero-crossing rate**, which measure how noisy, rather than tonal, a sound is;

- **attack time**, the time the note takes to reach its peak amplitude, which separates plucked strings from bowed and blown instruments.

We standardize each feature using statistics from the training set. We do not perform explicit feature selection, since the feature set is small and both models are reasonably robust to uninformative features.

CNN. The CNN takes the log-mel spectrogram directly as a single-channel image, with mel frequency on one axis and time on the other. It uses no hand-crafted features. Instead, the convolutional layers learn their own representation of the spectrogram, and the whole model, from spectrogram to class prediction, is trained end-to-end as one network. To reduce overfitting on our relatively small dataset, we plan to apply SpecAugment-style masking (Park et al., 2019), which hides random bands of frequencies and time steps during training. If the small CNN proves too limited, we can replace its learned representation with embeddings from a pretrained PANNs model (Kong et al., 2020).

5 Implementation

We implement three models of increasing complexity: a majority-class baseline, a classical baseline using the hand-crafted features from Section 4, and a convolutional neural network trained on log-mel spectrograms. Audio loading and feature extraction use librosa, the classical models use scikit-learn, and the CNN is implemented in PyTorch. All models are trained and evaluated on the same splits, described in Section 6.

Majority-class baseline. The simplest baseline always predicts the most frequent class in the training set, which is violin. Since violin makes up about 11% of the clips, this baseline reaches roughly 11% accuracy, but its balanced accuracy is only $1/19 \approx 5.3\%$, since it never predicts any other class. Any useful model must clearly beat both of these numbers, and the gap between them shows why accuracy alone is misleading on our imbalanced dataset.

Layer	Output channels
Input log-mel spectrogram	1
Conv block 1	[TODO: n]
Conv block 2	[TODO: n]
Conv block 3	[TODO: n]
Conv block 4	[TODO: n]
Global average pooling	[TODO: n]
Dropout ($p =$ [TODO: p]) + linear	19

Table 2: CNN architecture. Each conv block is a 3×3 convolution, batch normalization, ReLU, and 2×2 max pooling.

Classical baseline. Our second baseline trains two classifiers on the standardized feature vectors. The first is a support vector machine with a radial basis function (RBF) kernel, which minimizes the regularized hinge loss and handles the 19 classes by training one binary classifier per pair of classes. The second is a random forest, which grows each tree by choosing splits that minimize Gini impurity and predicts by majority vote over the trees. To account for class imbalance, both models weight each class in inverse proportion to its frequency in the training set. We tune the SVM's regularization strength C and kernel width γ, and the forest's number of trees and maximum depth, by grid search on the validation set. The selected values are $C =$ [TODO: value], $\gamma =$ [TODO: value], [TODO: number] trees, and a maximum depth of [TODO: value].

CNN. Our main model is a small CNN with four convolutional blocks, shown in Table 2. Each block applies a 3×3 convolution, batch normalization, a ReLU activation, and 2×2 max pooling, so each block halves the size of the spectrogram along both the frequency and time axes while increasing the number of channels. After the last block, global average pooling collapses the remaining frequency and time dimensions into a single vector, which also means the network does not depend on the exact clip length. A dropout layer and a fully connected layer then map this vector to 19 output scores, one per instrument, and a softmax turns these into class probabilities. In total, the network has [TODO: number] trainable parameters.

We train the CNN with a class-weighted cross-entropy loss. For a batch of N clips with labels y_i and predicted class probabilities $\hat{p}_i$, the loss is

$$\mathcal{L} = -\frac{1}{N} \sum_{i=1}^{N} w_{y_i} \log \hat{p}_{i,y_i}, \qquad w_c = \frac{N_{\text{train}}}{C \cdot N_c}, \tag{1}$$

where $C = 19$ is the number of classes and N_c is the number of training clips in class c. Without these weights, the loss would be dominated by the largest classes, and the model could largely ignore rare instruments such as banjo and mandolin.

We optimize the loss with Adam, using a learning rate of [TODO: value], weight decay of [TODO: value], and a batch size of [TODO: value]. We train for up to [TODO: number] epochs and keep the model from the epoch with the highest macro-F1 on the validation set, stopping early if validation macro-F1 does not improve for [TODO: number] epochs. During training, we apply SpecAugment-style frequency and time masking to each spectrogram, as described in Section 4. Training takes about [TODO: time] on [TODO: hardware].

Implementation details. Decoding and resampling over 13,000 MP3 files is slow, so we do it once and cache the processed log-mel spectrograms and feature vectors as NumPy arrays. This makes each training epoch much faster and ensures that every model sees exactly the same inputs. We fix the random seeds for the data split, model initialization, and data loading so that results are reproducible. [TODO: Describe any implementation problems the group ran into and how they were solved.]

Ablations. To understand which parts of the CNN matter, we plan to compare the full model against versions trained without class weighting, without SpecAugment, and with 1-second rather than 2-second clips. For the classical baseline, we will compare the random forest's feature importances to see which hand-crafted features are most useful for telling instruments apart. [TODO: Report ablation results here or in Section 6 once they are run.]

6 Results and Evaluation

How are you evaluating your model? What results do you have so far? What are your baselines? Refer to item 5. This may take around 0.5 pages.

7 Feedback and Plans

Write about your plans for the remainder of the project. This should include a discussion of the feedback you received from your TA, and how you plan to improve your approach. Reflect on your implementation and areas for improvement. Refer to item 6. This may be around 0.5 pages.

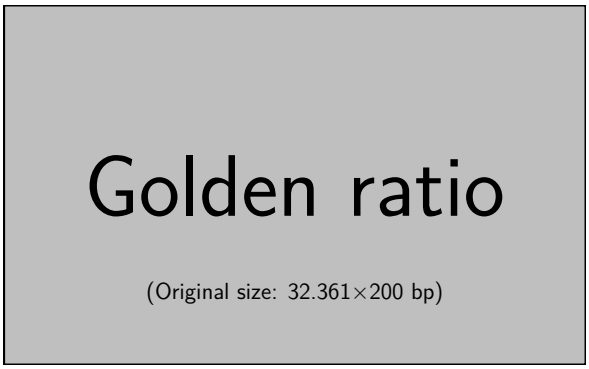


Figure 1: A figure with a caption that runs for more than one line. Example image is usually available through the `mwe` package without even mentioning it in the preamble.

8 Template Notes

You can remove this section or comment it out, as it only contains instructions for how to use this template. You may use subsections in your document as you find appropriate.

8.1 Tables and figures

See Table 3 for an example of a table and its caption. See Figure 1 for an example of a figure and its caption.

8.2 Citations

Table 3 shows the syntax supported by the style files. We encourage you to use the natbib styles. You can use the command `\citet` (cite in text) to get "author (year)" citations, like this citation to a paper by Gusfield (1997). You can use the command `\citep` (cite in parentheses) to get "(author, year)" citations (Gusfield, 1997). You can use the command `\citealp` (alternative cite without parentheses) to get "author, year" citations, which is useful for using citations within parentheses (e.g. Gusfield, 1997).

8.3 References

Many websites where you can find academic papers also allow you to export a bib file for citation or bib formatted entry. Copy this into the `custom.bib` and you will be able to cite the paper in the LaTeX. You can remove the example entries.

8.4 Equations

An example equation is shown below:

$$A = \pi r^2 \tag{2}$$

Labels for equation numbers, sections, subsections, figures and tables are all defined with the

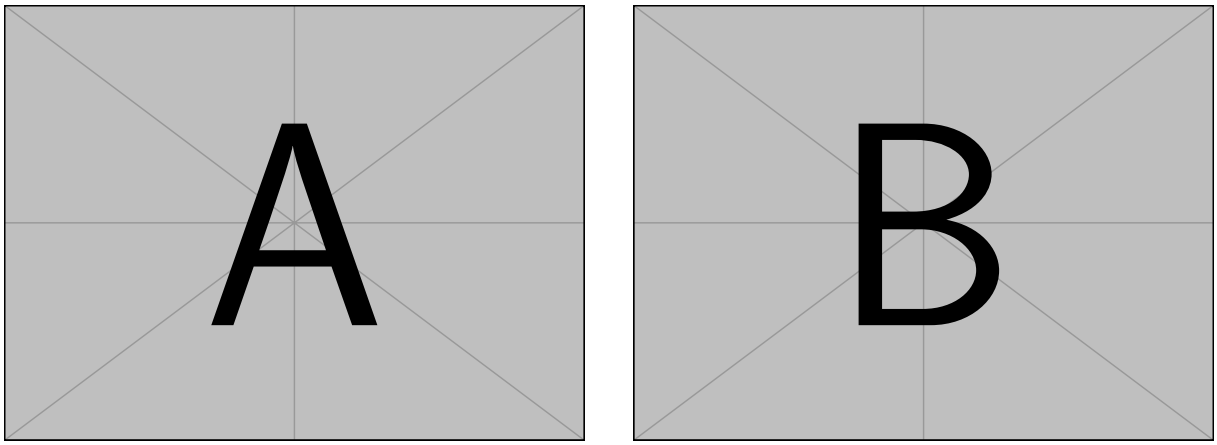


Figure 2: A minimal working example to demonstrate how to place two images side-by-side.

Output	natbib command	ACL only command
(Gusfield, 1997)	`\citep`	
Gusfield, 1997	`\citealp`	
Gusfield (1997)	`\citet`	
(1997)	`\citeyearpar`	
Gusfield's (1997)		`\citeposs`

Table 3: Citation commands supported by the style file.

`\label{label}` command and cross references to them are made with the `\ref{label}` command. This an example cross-reference to Equation 2. You can also write equations inline, like this: $A = \pi r^2$.

Team Contributions

Write in this section a few sentences describing the contributions of each team member. What did each member work on? Refer to item 7.

References

Rie Kubota Ando and Tong Zhang. 2005. A framework for learning predictive structures from multiple tasks and unlabeled data. *Journal of Machine Learning Research*, 6:1817–1853.

Galen Andrew and Jianfeng Gao. 2007. Scalable training of L1-regularized log-linear models. In *Proceedings of the 24th International Conference on Machine Learning*, pages 33–40.

Juan J. Bosch, Jordi Janer, Ferdinand Fuhrmann, and Perfecto Herrera. 2012. A comparison of sound segregation techniques for predominant instrument recognition in musical audio signals. In *Proceedings of the 13th International Society for Music Information Retrieval Conference*.

Jesse Engel, Cinjon Resnick, Adam Roberts, Sander Dieleman, Douglas Eck, Karen Simonyan, and Mohammad Norouzi. 2017. Neural audio synthesis of musical notes with WaveNet autoencoders. In *Proceedings of the 34th International Conference on Machine Learning*.

Antti Eronen and Anssi Klapuri. 2000. Musical instrument recognition using cepstral coefficients and temporal features. In *Proceedings of the IEEE International Conference on Acoustics, Speech, and Signal Processing*.

Dan Gusfield. 1997. *Algorithms on Strings, Trees and Sequences*. Cambridge University Press, Cambridge, UK.

Yoonchang Han, Jaehun Kim, and Kyogu Lee. 2017. Deep convolutional neural networks for predominant instrument recognition in polyphonic music. *IEEE/ACM Transactions on Audio, Speech, and Language Processing*, 25(1):208–221.

Perfecto Herrera-Boyer, Geoffroy Peeters, and Shlomo Dubnov. 2003. Automatic classification of musical instrument sounds. *Journal of New Music Research*, 32(1):3–21.

Qiuqiang Kong, Yin Cao, Turab Iqbal, Yuxuan Wang, Wenwu Wang, and Mark D. Plumbley. 2020. PANNs: Large-scale pretrained audio neural networks for audio pattern recognition. *IEEE/ACM Transactions on Audio, Speech, and Language Processing*, 28:2880–2894.

Daniel S. Park, William Chan, Yu Zhang, Chung-Cheng Chiu, Barret Zoph, Ekin D. Cubuk, and Quoc V. Le. 2019. SpecAugment: A simple data augmentation

method for automatic speech recognition. In *Proceedings of Interspeech*, pages 2613–2617.

Philharmonia Orchestra. n.d. Sound samples. https://philharmonia.co.uk/resources/sound-samples/. Accessed: 2026-11-02.

Mohammad Sadegh Rasooli and Joel R. Tetreault. 2015. Yara parser: A fast and accurate dependency parser. *Computing Research Repository*, arXiv:1503.06733. Version 2.